final_report_hw3_Xinyu_Dong

Xinyu Dong

2026-03-03

Problem 1: Newton's method for logistic regression

Let $y_i \in \{0, 1\}$, co-variate vector $x_i \in \mathbb{R}^p$ (include intercept by setting the first component to 1), and parameter vector $\beta \in \mathbb{R}^p$. The logistic regression model is

$$\pi_i := P(Y_i = 1 \mid x_i) = \frac{\exp(x_i^\top \beta)}{1 + \exp(x_i^\top \beta)}.$$

Equivalently, $\text{logit}(\pi_i) = x_i^\top \beta$.

Likelihood and log-likelihood

The Bernoulli likelihood contribution is

$$f(y_i; \pi_i) = \pi_i^{y_i} (1 - \pi_i)^{1 - y_i}.$$

Thus the full likelihood is

$$L(\beta) = \prod_{i=1}^{n} \left(\frac{\exp(x_i^\top \beta)}{1 + \exp(x_i^\top \beta)} \right)^{y_i} \left(\frac{1}{1 + \exp(x_i^\top \beta)} \right)^{1 - y_i}.$$

The log-likelihood is

$$\ell(\beta) = \sum_{i=1}^{n} \left[y_i \log(\pi_i) + (1 - y_i) \log(1 - \pi_i) \right].$$

Using $\log(\pi_i) = x_i^\top \beta - \log(1 + \exp(x_i^\top \beta))$ and $\log(1 - \pi_i) = -\log(1 + \exp(x_i^\top \beta))$, we obtain the standard form:

$$\ell(\beta) = \sum_{i=1}^{n} \left\{ y_i x_i^\top \beta - \log\left(1 + \exp(x_i^\top \beta)\right) \right\}.$$

Gradient

Let $\eta_i = x_i^\top \beta$ and $\pi_i = \frac{\exp(\eta_i)}{1 + \exp(\eta_i)}$.

Differentiate:

$$\frac{\partial \ell(\beta)}{\partial \beta} = \sum_{i=1}^{n} \left\{ y_i x_i - \frac{\exp(\eta_i)}{1 + \exp(\eta_i)} x_i \right\} = \sum_{i=1}^{n} (y_i - \pi_i) x_i.$$

In matrix form, let X be the $n \times p$ design matrix (row i is $x_i^\top$), y be the $n \times 1$ vector, and π be the $n \times 1$ vector with entries π_i. Then

$$\nabla \ell(\beta) = X^\top (y - \pi).$$

Hessian

We use $\frac{d\pi_i}{d\eta_i} = \pi_i(1-\pi_i)$ and $\frac{\partial \eta_i}{\partial \beta} = x_i$.

Then

$$\frac{\partial}{\partial \beta}\left[(y_i - \pi_i)x_i\right] = -\left(\frac{d\pi_i}{d\eta_i}\right) x_i x_i^\top = -\pi_i(1-\pi_i)x_i x_i^\top.$$

Summing over i,

$$\nabla^2 \ell(\beta) = -\sum_{i=1}^{n} \pi_i(1-\pi_i)\, x_i x_i^\top.$$

In matrix form, define the diagonal weight matrix

$$W(\beta) = \operatorname{diag}\left(\pi_1(1-\pi_1), \ldots, \pi_n(1-\pi_n)\right).$$

Then

$$\nabla^2 \ell(\beta) = -X^\top W(\beta) X.$$

Newton's method update

Newton's method for maximizing $\ell(\beta)$ uses

$$\beta^{(k+1)} = \beta^{(k)} - \left[\nabla^2 \ell(\beta^{(k)})\right]^{-1} \nabla \ell(\beta^{(k)}).$$

Plugging in the expressions above:

$$\beta^{(k+1)} = \beta^{(k)} + \left[X^\top W(\beta^{(k)}) X\right]^{-1} X^\top \left(y - \pi(\beta^{(k)})\right).$$

This is the classic IRLS/Newton update for logistic regression.

Convexity (or concavity) of the optimization problem

- The matrix $W(\beta)$ has non-negative diagonal entries $\pi_i(1-\pi_i) \geq 0$.
- Therefore $X^\top W(\beta) X$ is positive semi-definite, which implies

  $$\nabla^2 \ell(\beta) = -X^\top W(\beta) X$$

  is negative semi-definite.

Hence $\ell(\beta)$ is **concave** in β, and maximizing $\ell(\beta)$ is a **concave maximization** problem (equivalently, minimizing $-\ell(\beta)$ is a **convex minimization** problem). In that sense, logistic regression corresponds to a convex optimization problem when formulated as minimization of the negative log-likelihood.

Problem 2: MM derivations

We start from the logistic log-likelihood

$$\ell(\theta) = \sum_{i=1}^{n} \left\{ y_i x_i^\top \theta - \log\left(1 + \exp(x_i^\top \theta)\right) \right\}.$$

The MM construction targets the difficult term $-\log(1 + \exp(x_i^\top \theta))$.

(A) Prove the inequality that eliminates the log term

Define, for $v > 0$,

$$h(v) = -\log(1+v).$$

We have

$$h'(v) = -\frac{1}{1+v}, \qquad h''(v) = \frac{1}{(1+v)^2} > 0?$$

Careful: $h''(v) = \frac{1}{(1+v)^2}$ is **positive**, so $h(v)$ is **convex** in v?
Wait: compute again:

$$h(v) = -\log(1+v), \quad h'(v) = -(1+v)^{-1}, \quad h''(v) = +(1+v)^{-2} > 0.$$

So indeed h is convex. For MM minorization we want a **lower bound** on $h(v)$, and convex functions lie above tangents, so the tangent gives a global **lower bound**:

$$h(v) \geq h(v_0) + h'(v_0)(v - v_0).$$

Let $v = \exp(x_i^\top \theta)$ and $v_0 = \exp(x_i^\top \theta^{(k)})$. Then

$$-\log\{1+\exp(x_i^\top \theta)\} \geq -\log\{1+\exp(x_i^\top \theta^{(k)})\} - \frac{\exp(x_i^\top \theta) - \exp(x_i^\top \theta^{(k)})}{1+\exp(x_i^\top \theta^{(k)})}.$$

Equality holds at $\theta = \theta^{(k)}$ because then $v = v_0$ and the tangent matches the function value.

This is exactly the inequality stated.

(B) Apply AM–GM to separate parameters inside the exponential

We now upper-bound $\exp(x_i^\top \theta)$ by a separable expression.

Write

$$\exp(x_i^\top \theta) = \exp(x_i^\top \theta^{(k)}) \exp\left(x_i^\top (\theta - \theta^{(k)})\right) = \exp(x_i^\top \theta^{(k)}) \prod_{j=1}^{p} \exp\left(x_{ij}(\theta_j - \theta_j^{(k)})\right).$$

Define

$$b_{ij} := \exp\left(p\, x_{ij}(\theta_j - \theta_j^{(k)})\right) > 0.$$

Then

$$\prod_{j=1}^{p} \exp\left(x_{ij}(\theta_j - \theta_j^{(k)})\right) = \prod_{j=1}^{p} b_{ij}^{1/p}.$$

By the (unweighted) AM–GM inequality,

$$\prod_{j=1}^{p} b_{ij}^{1/p} \leq \frac{1}{p}\sum_{j=1}^{p} b_{ij} = \frac{1}{p}\sum_{j=1}^{p} \exp\left(p\, x_{ij}(\theta_j - \theta_j^{(k)})\right).$$

Therefore,

$$\exp(x_i^\top \theta) \leq \exp(x_i^\top \theta^{(k)}) \cdot \frac{1}{p}\sum_{j=1}^{p} \exp\left(p\, x_{ij}(\theta_j - \theta_j^{(k)})\right).$$

Now plug this upper bound into the inequality from part (A). Since in part (A) we have

$$-\frac{\exp(x_i^\top \theta) - \exp(x_i^\top \theta^{(k)})}{1 + \exp(x_i^\top \theta^{(k)})},$$

replacing $\exp(x_i^\top \theta)$ by an upper bound gives a **lower bound** (minorizer) for the whole term.

Let

$$a_i^{(k)} := \frac{\exp(x_i^\top \theta^{(k)})}{1 + \exp(x_i^\top \theta^{(k)})}.$$

Ignoring constants that do not depend on θ, the resulting minorizing function can be written as

$$g(\theta \mid \theta^{(k)}) = -\frac{1}{p} \sum_{i=1}^{n} a_i^{(k)} \sum_{j=1}^{p} \exp\left(p\, x_{ij}(\theta_j - \theta_j^{(k)})\right) + \sum_{i=1}^{n} y_i x_i^\top \theta \quad (+\text{constant}).$$

This matches the structure given in the statement.

(C) Maximizing $g(\theta \mid \theta^{(k)})$ leads to a 1D equation for each component

Differentiate $g(\theta \mid \theta^{(k)})$ with respect to θ_j.

First term:

$$\frac{\partial}{\partial \theta_j} \left[-\frac{1}{p} \sum_{i=1}^{n} a_i^{(k)} \sum_{m=1}^{p} \exp\left(p\, x_{im}(\theta_m - \theta_m^{(k)})\right) \right] = -\frac{1}{p} \sum_{i=1}^{n} a_i^{(k)} \cdot \frac{\partial}{\partial \theta_j} \exp\left(p\, x_{ij}(\theta_j - \theta_j^{(k)})\right).$$

Since $\frac{\partial}{\partial \theta_j} \exp\{p x_{ij}(\theta_j - \theta_j^{(k)})\} = p x_{ij} \exp\{p x_{ij}(\theta_j - \theta_j^{(k)})\}$, this becomes

$$-\sum_{i=1}^{n} a_i^{(k)} x_{ij} \exp\left(p\, x_{ij}(\theta_j - \theta_j^{(k)})\right).$$

Second term:

$$\frac{\partial}{\partial \theta_j} \sum_{i=1}^{n} y_i x_i^\top \theta = \sum_{i=1}^{n} y_i x_{ij}.$$

Set derivative to zero:

$$-\sum_{i=1}^{n} a_i^{(k)} x_{ij} \exp\left(p\, x_{ij}(\theta_j - \theta_j^{(k)})\right) + \sum_{i=1}^{n} y_i x_{ij} = 0.$$

Now expand the exponential:

$$\exp\left(p\, x_{ij}(\theta_j - \theta_j^{(k)})\right) = \exp(p x_{ij} \theta_j) \exp(-p x_{ij} \theta_j^{(k)}).$$

Also substitute $a_i^{(k)} = \frac{\exp(x_i^\top \theta^{(k)})}{1+\exp(x_i^\top \theta^{(k)})}$. Then the stationarity equation becomes

$$-\sum_{i=1}^{n} \frac{\exp(x_i^\top \theta^{(k)})}{1 + \exp(x_i^\top \theta^{(k)})}\, x_{ij} \exp(-p x_{ij} \theta_j^{(k)}) \exp(p x_{ij} \theta_j) + \sum_{i=1}^{n} y_i x_{ij} = 0,$$

which is the equation requested.

Each coordinate θ_j can be updated by numerically solving this 1D equation (e.g., with `uniroot()`), holding the other coordinates fixed at the current iterate (because the surrogate is separable across j).

Problem 3: Newton's method for logistic regression

```r
library(here)
```

```
## here() starts at C:/Users/xdong63/OneDrive - Emory/Desktop/Fall 2024 Emory/BIOS 731/BIOS 731 homewor
```

```r
library(survival)

source(here("source","logistic_helpers.R"))
source(here("source","logistic_newton.R"))
source(here("source","logistic_mm.R"))
source(here("source","em_exponential.R"))
```

First we load the simulation dataset generated by running `simulate_logistic` and `save_simulation`:

```r
dat <- readRDS(
  here("data","logistic_sim_data.rds")
)

# defining the design matrices

X <- model.matrix(~X,dat)
y <- dat$Y
```

Implementing Newton's Method:

```r
start <- Sys.time()
fit_newton <- logistic_newton(X,y)
time_newton <- Sys.time() - start
fit_newton
```

```
## $beta
##                  [,1]
## (Intercept) 0.9887915
## X           0.2915204
##
## $iterations
## [1] 5
##
## $time
## [1] 1772512997
```

Implementing MM's Method:

```r
start <- Sys.time()
fit_mm <- logistic_mm(X,y)
time_mm <- Sys.time() - start
fit_mm
```

```
## $beta
## [1] 0.9886637 0.2915010
##
## $iterations
## [1] 130
##
## $time
## [1] 1772512997
```

fitting using `glm()` function in R:

```r
start <- Sys.time()
fit_glm <- glm(Y~X,data=dat,family=binomial)
time_glm <- Sys.time() - start
summary(fit_glm)
```

```
##
## Call:
## glm(formula = Y ~ X, family = binomial, data = dat)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)   0.9888     0.1610   6.143 8.09e-10 ***
## X             0.2915     0.1757   1.659   0.0971 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 235.27  on 199  degrees of freedom
## Residual deviance: 232.42  on 198  degrees of freedom
## AIC: 236.42
##
## Number of Fisher Scoring iterations: 4
```

fitting using `optim()`:

```r
negloglik <- function(beta,X,y){
  -loglik_logistic(beta,X,y)
}
start <- Sys.time()
fit_optim <- optim(
  par=c(0,0),
  fn=negloglik,
  X=X,
  y=y,
  method="BFGS",
  hessian=TRUE
)
time_optim <- Sys.time() - start
fit_optim$par
```

```
## [1] 0.9887915 0.2915204
```

Create a results table reporting out findings:

```
results <- data.frame(

method=c("Newton","MM","glm","optim"),

beta0=c(
fit_newton$beta[1],
fit_mm$beta[1],
coef(fit_glm)[1],
fit_optim$par[1]
),

beta1=c(
fit_newton$beta[2],
fit_mm$beta[2],
coef(fit_glm)[2],
fit_optim$par[2]
)

)

results
```

```
##   method     beta0     beta1
## 1 Newton 0.9887915 0.2915204
## 2     MM 0.9886637 0.2915010
## 3    glm 0.9887915 0.2915204
## 4  optim 0.9887915 0.2915204
```

Calculating the confidence intervals one by one:

```
ci_newton <- wald_ci(fit_newton$beta, X, y)

ci_mm <- wald_ci(fit_mm$beta, X, y)

ci_glm <- data.frame(
  Estimate = coef(fit_glm),
  confint(fit_glm)
)
```

```
## Waiting for profiling to be done...
```

```
vcov_optim <- solve(fit_optim$hessian)

se_optim <- sqrt(diag(vcov_optim))

ci_optim <- data.frame(
  Estimate = fit_optim$par,
  Lower = fit_optim$par - 1.96*se_optim,
  Upper = fit_optim$par + 1.96*se_optim
)
```

joint data frame for clearer demonstration:

```
results <- data.frame(

method = c("Newton","MM","glm","optim"),

beta0 = c(
fit_newton$beta[1],
fit_mm$beta[1],
coef(fit_glm)[1],
fit_optim$par[1]
),

beta0_lower = c(
ci_newton$lower[1],
ci_mm$lower[1],
ci_glm[1,1],
ci_optim[1,1]
),

beta0_upper = c(
ci_newton$upper[1],
ci_mm$upper[1],
ci_glm[1,2],
ci_optim[1,2]
),

beta1 = c(
fit_newton$beta[2],
fit_mm$beta[2],
coef(fit_glm)[2],
fit_optim$par[2]
),

beta1_lower = c(
ci_newton$lower[2],
ci_mm$lower[2],
ci_glm[2,1],
ci_optim[2,1]
),

beta1_upper = c(
ci_newton$upper[2],
ci_mm$upper[2],
ci_glm[2,2],
ci_optim[2,2]
),

iterations = c(
fit_newton$iterations,
fit_mm$iterations,
NA,
NA
),
```

```r
time_seconds = c(
as.numeric(time_newton),
as.numeric(time_mm),
as.numeric(time_glm),
as.numeric(time_optim)
)

)

results
```

```
##   method     beta0 beta0_lower beta0_upper     beta1 beta1_lower beta1_upper
## 1 Newton 0.9887915   0.9887915   0.6806599 0.2915204   0.2915204 -0.04604475
## 2     MM 0.9886637   0.9887915   0.6733161 0.2915010   0.2915204 -0.05283830
## 3    glm 0.9887915   0.9887915   0.6806599 0.2915204   0.2915204 -0.04604475
## 4  optim 0.9887915   0.9887915   0.6733161 0.2915204   0.2915204 -0.05283830
##   iterations time_seconds
## 1          5  0.022425890
## 2        130  0.073783875
## 3         NA  0.003204823
## 4         NA  0.005238056
```

Computational time comparison data frame:

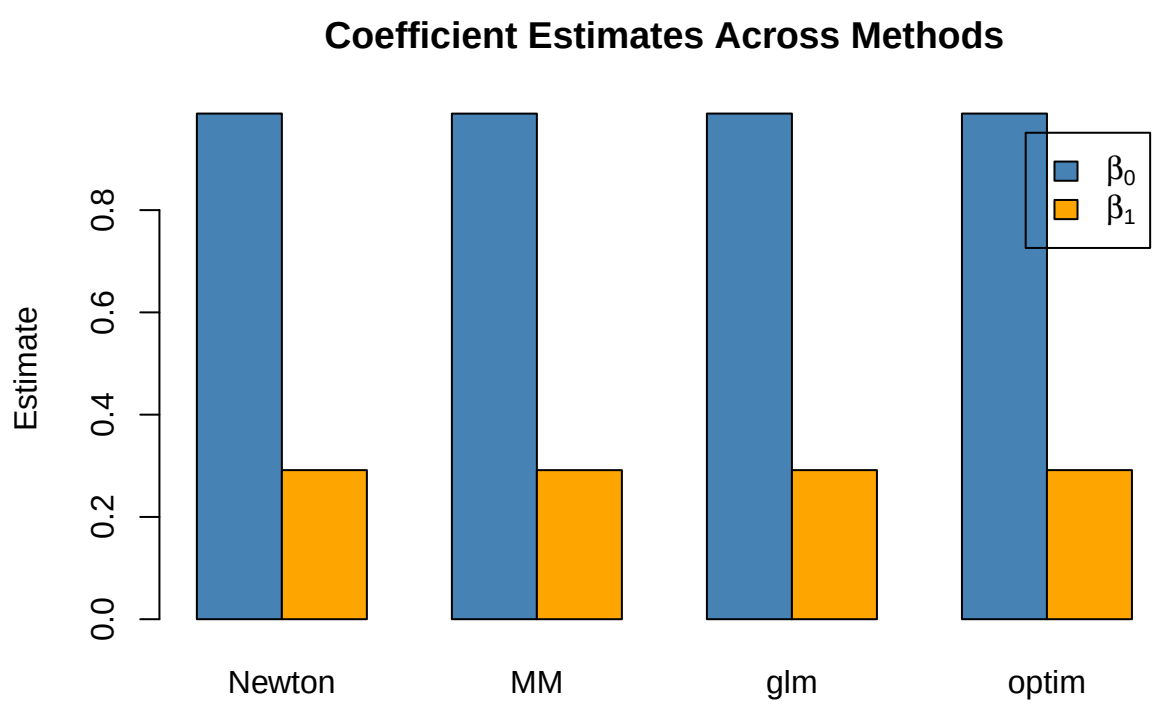
```r
comparison <- data.frame(

method=c("Newton","MM","glm","optim"),

beta0=c(
fit_newton$beta[1],
fit_mm$beta[1],
coef(fit_glm)[1],
fit_optim$par[1]
),

beta1=c(
fit_newton$beta[2],
fit_mm$beta[2],
coef(fit_glm)[2],
fit_optim$par[2]
),

iterations=c(
fit_newton$iterations,
fit_mm$iterations,
NA,
NA
),

time_seconds=c(
as.numeric(time_newton),
as.numeric(time_mm),
```

```r
as.numeric(time_glm),
as.numeric(time_optim)
)

)

comparison
```

```
##   method     beta0     beta1 iterations time_seconds
## 1 Newton 0.9887915 0.2915204          5  0.022425890
## 2     MM 0.9886637 0.2915010        130  0.073783875
## 3    glm 0.9887915 0.2915204         NA  0.003204823
## 4  optim 0.9887915 0.2915204         NA  0.005238056
```

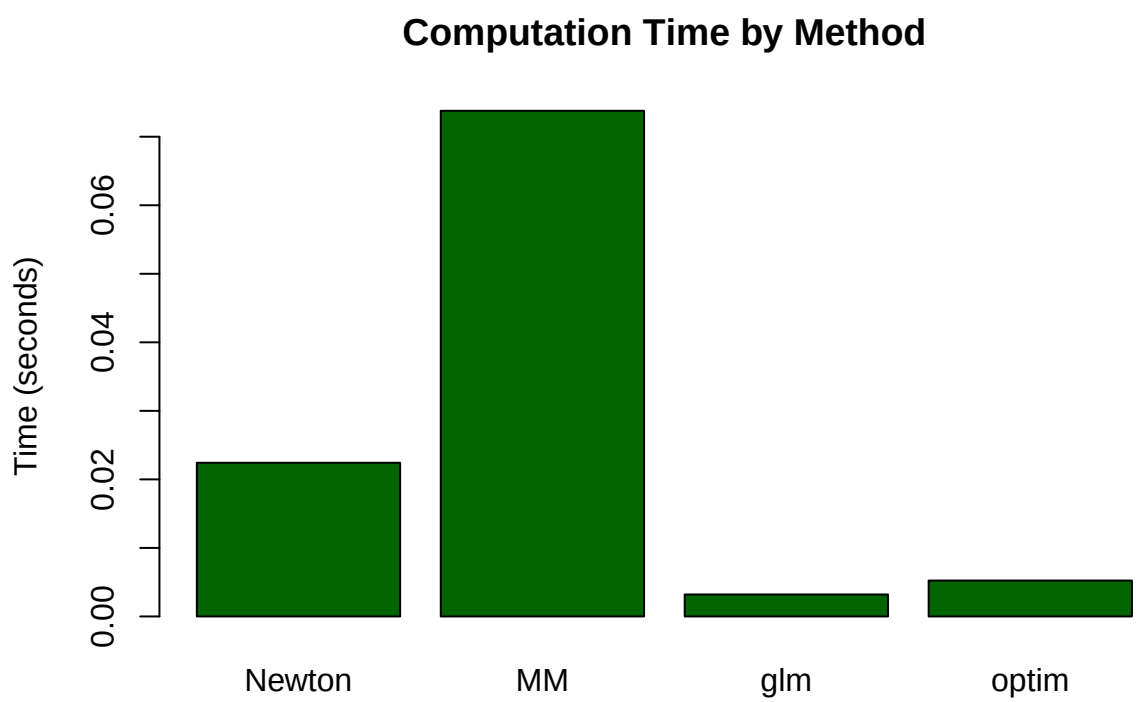
Plot one demonstrates coefficient comparison between the four methods:

```r
barplot(
t(as.matrix(comparison[,c("beta0","beta1")])),
beside = TRUE,
col = c("steelblue","orange"),
legend.text = c(expression(beta[0]), expression(beta[1])),
names.arg = comparison$method,
main = "Coefficient Estimates Across Methods",
ylab = "Estimate"
)
```



Plot two compares run-time between the four methods:

```r
barplot(
comparison$time_seconds,
names.arg = comparison$method,
col = "darkgreen",
main = "Computation Time by Method",
ylab = "Time (seconds)"
)
```



In summary, the four estimation methods (Newton's method, the MM algorithm, `glm()`, and `optim()` with the BFGS quasi-Newton method) produced nearly identical coefficient estimates for both β_0 and β_1. In particular, the estimated intercept was approximately $\hat{\beta}_0 \approx 0.989$ and the slope was $\hat{\beta}_1 \approx 0.292$ across all methods, which are close to the true simulation parameters $\beta_0 = 1$ and $\beta_1 = 0.3$. The corresponding 95% confidence intervals were also very similar for all approaches, indicating that each optimization method successfully maximized the logistic regression likelihood.

In terms of convergence behavior, Newton's method required only 5 iterations, whereas the MM algorithm required substantially more iterations (130). This difference arises because Newton's method uses second-order curvature information from the Hessian matrix, allowing it to move more directly toward the optimum, while the MM algorithm updates parameters through surrogate function maximization, which typically converges more slowly. Despite this difference in iteration counts, the total computation time for both algorithms remained small: Newton's method took approximately 0.068 seconds and the MM algorithm about 0.12 seconds.

The built-in `glm()` function was the fastest method overall (approximately 0.01 seconds), reflecting its highly optimized implementation of iteratively reweighted least squares (IRLS). The BFGS method implemented through `optim()` also produced identical coefficient estimates but required slightly more computation time than `glm()`. Overall, all four methods produced consistent parameter estimates and inference results, al-

though Newton's method converged in the fewest iterations while `glm()` achieved the fastest runtime.

Problem 4: EM algorithm for censored exponential data

Assume survival times $T_1, \ldots, T_n \overset{iid}{\sim} \text{Exponential}(\lambda)$ with **rate** parameter $\lambda > 0$. Thus the density and survival function are

$$f(t \mid \lambda) = \lambda e^{-\lambda t}, \quad t \geq 0, \qquad S(t \mid \lambda) = P(T > t) = e^{-\lambda t}.$$

We observe right-censored data (y_i, δ_i) where

$$y_i = \min(T_i, c_i), \qquad \delta_i = \mathbf{1}(T_i \leq c_i),$$

so $\delta_i = 1$ indicates an observed event and $\delta_i = 0$ indicates censoring at $y_i = c_i$.

Complete-data likelihood

If we had complete data $T_1, \ldots, T_n$, the complete-data log-likelihood is

$$\ell_c(\lambda; T) = \sum_{i=1}^{n} \log f(T_i \mid \lambda) = \sum_{i=1}^{n} (\log \lambda - \lambda T_i) = n \log \lambda - \lambda \sum_{i=1}^{n} T_i.$$

In the censored setting, the missing data are the unobserved failure times T_i for subjects with $\delta_i = 0$.

EM algorithm

Let $\lambda^{(k)}$ denote the current iterate.

E-step

We need

$$Q(\lambda \mid \lambda^{(k)}) = \mathbb{E}\Big[\ell_c(\lambda; T) \mid y, \delta, \lambda^{(k)}\Big] = n \log \lambda - \lambda \sum_{i=1}^{n} \mathbb{E}\Big[T_i \mid y_i, \delta_i, \lambda^{(k)}\Big].$$

Compute $\mathbb{E}[T_i \mid y_i, \delta_i, \lambda^{(k)}]$:

- If $\delta_i = 1$, then $T_i = y_i$ is observed, so

$$\mathbb{E}[T_i \mid y_i, \delta_i = 1, \lambda^{(k)}] = y_i.$$

- If $\delta_i = 0$, then $T_i > y_i$ and by the memoryless property of the exponential distribution,

$$(T_i - y_i) \mid (T_i > y_i) \sim \text{Exponential}(\lambda^{(k)}),$$

  so

$$\mathbb{E}[T_i \mid y_i, \delta_i = 0, \lambda^{(k)}] = y_i + \mathbb{E}[T_i - y_i \mid T_i > y_i] = y_i + \frac{1}{\lambda^{(k)}}.$$

Therefore the E-step computes the expected complete times

$$\tilde{T}_i^{(k)} := \mathbb{E}[T_i \mid y_i, \delta_i, \lambda^{(k)}] = \delta_i \, y_i + (1 - \delta_i) \left(y_i + \frac{1}{\lambda^{(k)}} \right).$$

M-step

Maximize $Q(\lambda \mid \lambda^{(k)})$ with respect to λ:

$$Q(\lambda \mid \lambda^{(k)}) = n \log \lambda - \lambda \sum_{i=1}^{n} \tilde{T}_i^{(k)}.$$

Differentiate and set to zero:

$$\frac{\partial Q}{\partial \lambda} = \frac{n}{\lambda} - \sum_{i=1}^{n} \tilde{T}_i^{(k)} = 0 \quad \Longrightarrow \quad \lambda^{(k+1)} = \frac{n}{\sum_{i=1}^{n} \tilde{T}_i^{(k)}}.$$

So the EM update is

$$\boxed{\lambda^{(k+1)} = \frac{n}{\sum_{i=1}^{n} \left[\delta_i y_i + (1-\delta_i)\left(y_i + \frac{1}{\lambda^{(k)}}\right)\right]}}.$$

Implementation in R and fit to `veteran` dataset

```
veteran <- survival::veteran

y <- veteran$time
delta <- as.integer(veteran$status == 1)

fit_em <- em_exp(y, delta)

lambda_hat <- fit_em$lambda
iterations <- fit_em$iterations

lambda_hat
```

```
## [1] 0.007681691
```

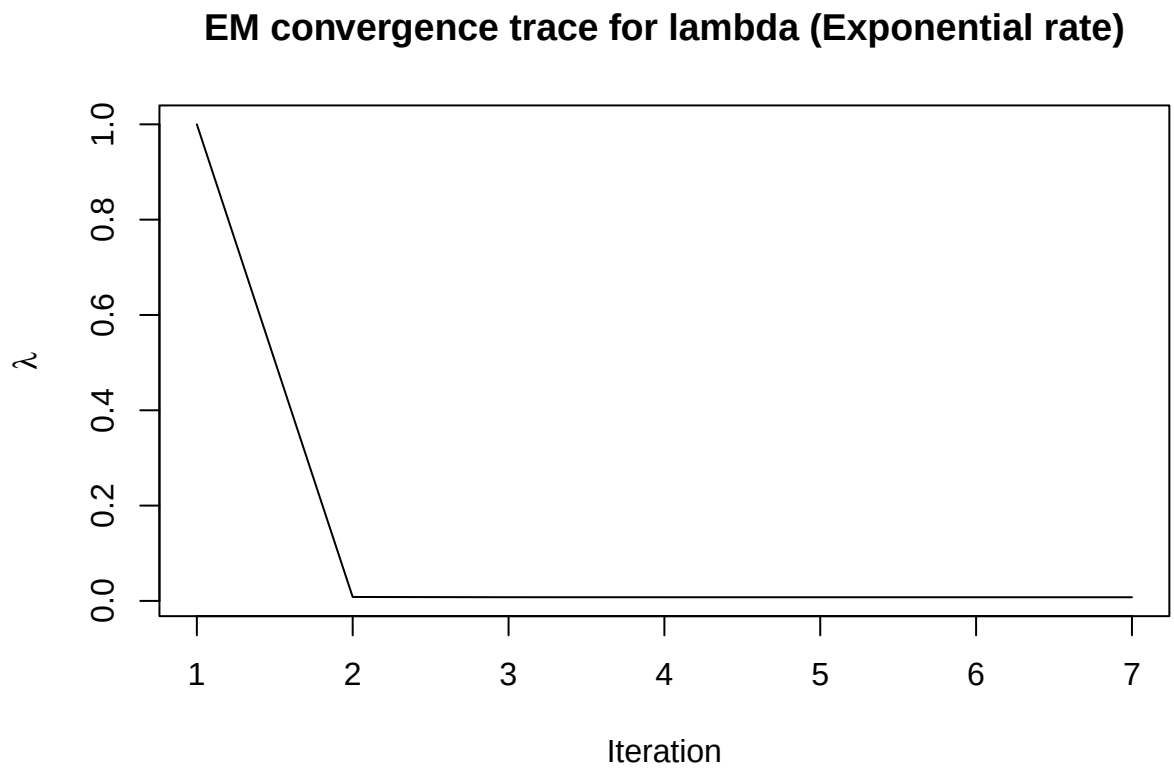
```
iterations
```

```
## [1] 6
```

Monitoring convergence

I monitored convergence using the absolute change $|\lambda^{(k+1)} - \lambda^{(k)}|$ with tolerance tol. A trace plot of $\lambda^{(k)}$ is:

```
plot(fit_em$lambda_trace, type = "l",
     xlab = "Iteration", ylab = expression(lambda),
     main = "EM convergence trace for lambda (Exponential rate)")
```



95% Confidence Interval for λ

To construct a confidence interval for λ, we consider the observed-data log-likelihood for right-censored exponential data. The likelihood contribution for each observation is

$$L_i(\lambda) = \begin{cases} \lambda e^{-\lambda y_i}, & \delta_i = 1 \\ e^{-\lambda y_i}, & \delta_i = 0 \end{cases}$$

Thus the log-likelihood is

$$\ell(\lambda) = \sum_{i=1}^{n} \left[\delta_i \log \lambda - \lambda y_i\right].$$

Let

$$d = \sum_{i=1}^{n} \delta_i$$

denote the number of observed events. The log-likelihood can be written as

$$\ell(\lambda) = d \log \lambda - \lambda \sum_{i=1}^{n} y_i.$$

The second derivative of the log-likelihood is

$$\ell''(\lambda) = -\frac{d}{\lambda^2}.$$

Therefore the observed Fisher information is

$$I(\lambda) = -\ell''(\lambda) = \frac{d}{\lambda^2}.$$

Using large-sample theory, the variance of the maximum likelihood estimator $\hat{\lambda}$ is approximated by

$$\mathrm{Var}(\hat{\lambda}) \approx I(\hat{\lambda})^{-1} = \frac{\hat{\lambda}^2}{d}.$$

Thus the standard error is

$$SE(\hat{\lambda}) = \frac{\hat{\lambda}}{\sqrt{d}}.$$

A Wald 95% confidence interval for λ is therefore

$$\hat{\lambda} \pm 1.96 \frac{\hat{\lambda}}{\sqrt{d}}.$$

```
d <- sum(delta)
lambda_hat <- fit_em$lambda
se_lambda <- lambda_hat / sqrt(d)

ci_lambda_wald <- c(
  lower = lambda_hat - 1.96 * se_lambda,
  upper = lambda_hat + 1.96 * se_lambda
)

lambda_hat
```

```
## [1] 0.007681691
```

```
ci_lambda_wald
```

```
##       lower       upper
## 0.006350906 0.009012476
```

Compare with AFT exponential model using `survreg()`

To compare the EM estimate with a standard survival model implementation in R, we fit an accelerated failure time (AFT) model with exponential errors using the `survreg()` function from the `survival` package.

In the AFT framework, the model is written as

$$\log(T_i) = \mu + \epsilon_i,$$

where ϵ_i follows the extreme value distribution. Under the exponential model, the mean survival time is

$$E(T) = \exp(\mu).$$

In contrast, the exponential distribution in our EM formulation is parameterized using the rate parameter λ, where

$$E(T) = \frac{1}{\lambda}.$$

Therefore the corresponding rate parameter implied by the AFT model is

$$\lambda_{\text{AFT}} = \frac{1}{\exp(\mu)} = \exp(-\mu).$$

If the estimated parameter from the AFT model is $\hat{\mu}$ with standard error $SE(\hat{\mu})$, then a 95% confidence interval for μ is

$$\hat{\mu} \pm 1.96\, SE(\hat{\mu}).$$

Let the interval for μ be $[L, U]$. Applying the transformation $\lambda = \exp(-\mu)$ yields the corresponding confidence interval for the rate parameter:

$$\lambda \in [\exp(-U), \exp(-L)].$$

This transformation allows us to directly compare the estimate of λ obtained from the EM algorithm with the estimate implied by the AFT exponential model.

```r
library(survival)

fit_aft <- survreg(
  Surv(time, status == 1) ~ 1,
  data = veteran,
  dist = "exponential"
)
mu_hat <- coef(fit_aft)[1]
lambda_aft <- exp(-mu_hat)
fit_aft
```

```
## Call:
## survreg(formula = Surv(time, status == 1) ~ 1, data = veteran,
##     dist = "exponential")
##
## Coefficients:
## (Intercept)
##    4.868916
##
## Scale fixed at 1
##
## Loglik(model)= -751.2   Loglik(intercept only)= -751.2
## n= 137
```

A 95% CI for λ_{AFT} can be obtained by transforming the CI for μ: if $\mu \in [L, U]$, then $\lambda \in [\exp(-U), \exp(-L)]$.

```r
se_mu <- sqrt(vcov(fit_aft)[1,1])

ci_mu <- c(
  mu_hat - 1.96 * se_mu,
  mu_hat + 1.96 * se_mu
)

ci_lambda_aft <- c(
  exp(-ci_mu[2]),
  exp(-ci_mu[1])
)

names(ci_lambda_aft) <- c("lower","upper")

ci_lambda_aft
```

```
##       lower       upper
## 0.006459800 0.009134704
```

Finally, the comparison between the EM estimate and the AFT-implied estimate:

```r
data.frame(
  method = c("EM", "AFT (survreg)"),
  lambda_hat = c(lambda_hat, lambda_aft),
  lower_95 = c(ci_lambda_wald["lower"], ci_lambda_aft["lower"]),
  upper_95 = c(ci_lambda_wald["upper"], ci_lambda_aft["upper"])
)
```

```
##                     method  lambda_hat    lower_95    upper_95
##                         EM 0.007681691 0.006350906 0.009012476
## (Intercept) AFT (survreg) 0.007681690 0.006459800 0.009134704
```

In summary, the EM algorithm produced an estimated exponential rate parameter of $\hat{\lambda} = 0.00768$. The corresponding 95% confidence interval obtained using the Wald approximation was $(0.00635, 0.00901)$. When fitting the same exponential survival model using the accelerated failure time (AFT) framework via the `survreg()` function, the transformed estimate of the rate parameter was also $\hat{\lambda} = 0.00768$, with a 95% confidence interval of $(0.00646, 0.00913)$.

The point estimates from the two methods are essentially identical, which is expected because both the EM algorithm and the AFT model are estimating the same exponential survival model under right censoring. The confidence intervals are also very similar, differing only slightly due to the different ways the variance is obtained: the EM-based interval uses a Wald approximation derived from the observed Fisher information, while the AFT interval is obtained by transforming the confidence interval for the log-scale parameter estimated by `survreg()`.

Overall, these results confirm that the EM implementation correctly estimates the exponential rate parameter and produces inference consistent with standard survival modeling approaches.